

RI.UVT.RO Redesign

Role 6 Week 4 Work Plan

Role: WordPress Developer — API & Content Types

Week: 4 **Generated:** 2026-05-12

Sources: `plan.pdf · end_changes.pdf · gap_report.pdf · week-4-todo.md`

Context Entering Week 4

The Week 3 gap report confirmed that Role 6 produced no documentation last week. The plugins (ACF and Custom Post Type UI) are installed and now tracked by git, but no CPTs, taxonomies, field groups, or backend planning documents were created.

This week carries a dual burden: **produce the missing Week 3 documentation first**, then proceed with WordPress configuration. Nothing that depends on those documents should be built until they exist.

Current state of the WordPress backend

Area	Status
ACF plugin	In wp-content/plugins/ — not yet activated
Custom Post Type UI plugin	In wp-content/plugins/ — not yet activated
CPTs (call, story, resource, programme, people)	Not created
Taxonomies	Not created
ACF field groups	Not created
REST API endpoints beyond WP default	Not implemented
Role fork folder structure	WordPress at repo root — must be fixed

Priority Order This Week

1. Fix repo folder structure
2. Write carry-over documentation (blocks everything else)
3. Activate plugins in WordPress admin
4. Register CPTs and taxonomies
5. Configure ACF field groups
6. Verify REST API responses
7. Create test entries for Role 5

Do not skip to step 3 before steps 1 and 2 are done. Role 5 cannot build API helpers without the REST API exposure checklist, and that document must exist before field naming is finalised.

Step 1 — Fix Folder Structure in Role Fork

This is a housekeeping task that must be done before the next PR.

Current structure, which is wrong:

```
roles/role6/ro.uvt.ri/  
  wp-admin/  
  wp-content/  
  wp-includes/  
  frontend/  
  docs/
```

Required structure, matching main:

```
roles/role6/ro.uvt.ri/  
  wordpress/  
    ro.uvt.ri/  
      wp-content/  
  frontend/  
  docs/
```

Move the WordPress files into `wordpress/ro.uvt.ri/`. This does not affect the local Laragon setup — Laragon reads from `C:/laragon/www/`, not from the repo folder.

Also pull the updated `.gitignore` from main into the role fork before the next commit.

Step 2 — Week 3 Carry-Over Documentation

These three documents were required last week and are now **priority-zero** for Week 4. They must be written before CPT configuration begins, because they define the contract that Role 5 will build API helpers against.

2a. [wordpress-content-structure-notes.md](#)

Document which content lives where. Use `06_WordPress_Content_Model.md` and the sitemap as sources. Cover:

- Which routes are backed by **WP Pages** (evergreen, static content)
- Which routes are backed by **WP Posts** (news, announcements)

- Which routes are backed by each **CPT** (calls, stories, resources, programmes, people)
- How **taxonomies** relate to each CPT
- Any content that does not fit neatly into the above categories

Reference structure from 06_WordPress_Content_Model.md:

Content type	Slug	Backed by
About	/about	WP Page
Erasmus section pages	/erasmus/*	WP Page
International Students section pages	/international-students/	WP Page
Partnerships	/partnerships	WP Page
News	/news	WP Post
Announcements	(under news)	WP Post
Open calls	/calls	CPT call
Student stories	/stories	CPT story
Downloadable resources	/resources	CPT resource
Study programmes	/programmes	CPT programme
Staff / contacts	(TBD)	CPT people — confirm with Role 1

2b. [rest-api-exposure-checklist.md](#)

List every endpoint the frontend will consume. For each endpoint, specify:

- The endpoint URL
- Whether it is a default WP endpoint or requires a registered CPT
- Which fields must appear in the REST response
- Whether ACF fields need explicit REST API exposure enabled
- Any filters or query parameters the frontend will use, such as `?audience=incoming-students`

Minimum endpoints to document:

```
/wp-json/wp/v2/pages
/wp-json/wp/v2/posts
/wp-json/wp/v2/calls
/wp-json/wp/v2/stories
/wp-json/wp/v2/resources
/wp-json/wp/v2/programmes
/wp-json/wp/v2/people      (if approved)
```

For each CPT endpoint, list the ACF fields that must appear in the response. This is the document Role 5 uses to write `getProgrammes()`, `getCalls()`, `getStories()`, and `getResources()` in `api/wordpress.js`.

Coordinate with Role 5 before finalising field names. Once agreed, field names must not change — renaming breaks the frontend API helpers.

2c. `static-vs-dynamic-content.md`

Classify every route in the sitemap as static or dynamic.

Classification	Meaning	Example
Static	Content lives in a WP Page. Rarely changes. Frontend fetches by slug.	About page, Erasmus overview
Dynamic — list	Frontend queries a CPT archive. Returns multiple entries.	<code>/calls</code> , <code>/programmes</code>
Dynamic — detail	Frontend fetches a single CPT entry by slug or ID.	<code>/calls/erasmus-2026</code>
Mixed	Page shell is static (WP Page) but includes dynamic blocks (CPT queries).	Homepage, Erasmus landing

This document is needed by Role 1 for consistency validation and by Role 2 for UX validation. Produce it as a simple table covering all routes in `01_Sitemap.md`.

Step 3 — Activate Plugins in WordPress Admin

Once the documentation is written:

1. Open Laragon — start Apache and MySQL
2. Navigate to <http://ro.uvt.ri.test/wp-admin>
3. Go to **Plugins** → **Installed Plugins**
4. Activate **Advanced Custom Fields**
5. Activate **Custom Post Type UI**

Step 4 — Register Custom Post Types via CPTUI

Create the following CPTs. For each one, **REST API support must be enabled** (`show_in_rest: true`) — without this, the frontend cannot consume the data.

CPT slug	Label	Has archive	REST enabled
call	Calls	Yes	Required
story	Stories	Yes	Required
resource	Resources	Yes	Required
programme	Programmes	Yes	Required
people	People	Optional	Required if approved

Before creating `people`: confirm with Role 1 whether this CPT is needed in Week 4 or can be deferred.

Recommended CPTUI settings for each CPT:

- Public: Yes
- Has archive: Yes, except `people` — confirm
- Show in REST: Yes
- REST API base: leave as default, matching the slug
- Supports: title, editor, thumbnail, excerpt

Step 5 — Register Taxonomies via CPTUI

Taxonomy slug	Label	Attached to
audience	Audience	All CPTs
programme-family	Programme Family	programme
content-topic	Content Topic	resource, story
academic-year	Academic Year	call, resource

For each taxonomy:

- Show in REST: Yes, required for frontend filtering
- Hierarchical: Yes for `programme-family` and `academic-year`; No for `audience` and `content-topic`

Step 6 — Create ACF Field Groups

Create one field group per CPT. The field group must have **Show in REST API** enabled, otherwise ACF field values will not appear in the `/wp-json/wp/v2/{cpt}` response.

Calls field group

Field label	Field name	Type
Deadline	deadline	Date picker
Eligibility	eligibility	Textarea
Application steps	application_steps	Repeater → Text sub-field
Documents	documents	Repeater → Label (text) + File URL (url)

Programmes field group

Field label	Field name	Type
Duration	duration	Text
Language	language	Text
Partner institution	partner_institution	Text
Application deadline	application_deadline	Date picker

Resources field group

Field label	Field name	Type
File URL	file_url	URL
File type	file_type	Select (PDF / DOCX / XLS / Other)
Audience notes	audience_notes	Textarea

Stories field group

Field label	Field name	Type
Author	author	Text
Story date	story_date	Date picker
Pull quote	pull_quote	Textarea

Global / shared field groups for pages

These are used across WP Pages rather than CPTs:

Group	Fields
CTA block	Heading (text), body (textarea), button label (text), button URL (url)
Process steps	Step repeater → step title (text) + step description (textarea)
FAQ section	FAQ repeater → question (text) + answer (textarea)
Contact card	Name (text), role (text), email (email), phone (text)
Document repeater	Label (text) + file URL (url) — reusable across page types

Step 7 — Verify REST API Responses

After activating plugins and creating CPTs and field groups, verify that each endpoint returns the expected data:

```
http://ro.uvt.ri.test/wp-json/wp/v2/calls
http://ro.uvt.ri.test/wp-json/wp/v2/stories
http://ro.uvt.ri.test/wp-json/wp/v2/resources
http://ro.uvt.ri.test/wp-json/wp/v2/programmes
```

Check that:

- The endpoint returns a 200 response
- The `acf` key appears in each entry's JSON, confirming ACF REST exposure is active
- Taxonomy terms appear in the response, confirming taxonomy REST exposure is active
- Field names match what was agreed in `rest-api-exposure-checklist.md`

If ACF fields do not appear: go to **ACF** → **Field Groups** → **[group name]** → **Settings** and enable **Show in REST API**.

Step 8 — Create Test Entries

Add at least one published entry per CPT so Role 5 can test their API helpers against real data. The entries do not need real content — placeholder values are fine. What matters is that all fields are populated so the frontend can validate the response shape.

Deliverables Checklist

Week 3 carry-overs: must be done first

- ☐ `wordpress-content-structure-notes.md` — content type assignment per route

- ❑ `rest-api-exposure-checklist.md` — every endpoint, expected fields, ACF exposure status
- ❑ `static-vs-dynamic-content.md` — route classification table

Week 4 implementation deliverables

- ❑ Role fork folder structure fixed (`wordpress/ro.uvt.ri/` in correct location)
- ❑ ACF and CPTUI activated in WordPress admin
- ❑ `call` CPT registered with REST support
- ❑ `story` CPT registered with REST support
- ❑ `resource` CPT registered with REST support
- ❑ `programme` CPT registered with REST support
- ❑ `people` CPT decision made with Role 1; registered if approved
- ❑ `audience` taxonomy registered and attached to all CPTs
- ❑ `programme-family` taxonomy registered
- ❑ `content-topic` taxonomy registered
- ❑ `academic-year` taxonomy registered
- ❑ ACF field group for `calls` created with REST enabled
- ❑ ACF field group for `programmes` created with REST enabled
- ❑ ACF field group for `resources` created with REST enabled
- ❑ ACF field group for `stories` created with REST enabled
- ❑ Global field groups created (CTA, process steps, FAQ, contact card, document repeater)
- ❑ All CPT endpoints verified returning correct JSON with ACF fields
- ❑ At least one test entry per CPT created and published
- ❑ `wordpress-cpt-taxonomy-setup.md` — record of CPT and taxonomy configuration
- ❑ `structured-fields-setup.md` — record of ACF field groups and field names

Communication Required This Week

Action	With whom	When
Confirm whether people CPT is needed	Role 1	Before creating it
Share <code>rest-api-exposure-checklist.md</code> for review	Role 5	As early as possible — unblocks API helpers
Agree on ACF field names before finalising	Role 5	Before step 6
Share <code>static-vs-dynamic-content.md</code>	Role 1 + Role 2	Once written

Definition of Done for Role 6 — Week 4

Week 4 is complete for Role 6 when all of the following are true:

- ☐ Three carry-over documents written and submitted
- ☐ All four CPTs registered with REST API support enabled
- ☐ All four taxonomies registered
- ☐ ACF field groups created for all CPTs with REST enabled
- ☐ All CPT REST endpoints verified and returning ACF fields
- ☐ At least one test entry per CPT exists in the local WordPress database
- ☐ Configuration documented in `wordpress-cpt-taxonomy-setup.md` and `structured-fields-setup.md`
- ☐ Role fork folder structure matches main

Week 5 Handoff Condition

Role 5 cannot begin content integration in Week 5 without:

1. The `rest-api-exposure-checklist.md` — needed to write typed API helpers
2. Confirmed CPT REST endpoints returning real data — needed to test those helpers
3. At least one test entry per CPT — needed to validate response shape

These are Role 6's responsibility. Delivering them by end of Week 4 is the critical path item for the entire project entering Week 5.